

02강

머신 러닝

파이썬 응용

통계·데이터과학과 박준희



✦ 학습목차

- 1 반복자 이해하기
- 2 함수의 개념과 구조
- 3 함수 응용 예제
- 4 클래스



1

반복자 이해하기



1 반복자 이해하기

● 반복 가능 객체(iterable)와 반복자(iterator)

- **iterable** : 하나씩 요소를 추출할 수 있는 객체
리스트, 튜플, 문자열, 딕셔너리 등이 해당
- **iterator** : **iterable** 객체에서 요소를 하나씩 꺼내기 위한 객체
next 함수로 요소를 하나씩 꺼내 사용 가능
메모리 효율적 방식으로 대용량 데이터 처리 시 유용

```
[16]: my_list = [1,2,3]
```

```
[17]: my_iterator = iter(my_list)
```

1

반복자 이해하기

● 반복 가능 객체(iterable)와 반복자(iterator)

```
[18]: next(my_iterator)
```

```
[18]: 1
```

```
[19]: next(my_iterator)
```

```
[19]: 2
```

```
[20]: next(my_iterator)
```

```
[20]: 3
```

1

반복자 이해하기

● 반복 가능 객체(iterable)와 반복자(iterator)

```
[21]: next(my_iterator)
```

```
-----  
StopIteration
```

```
Cell In[21], line 1
```

```
----> 1 next(my_iterator)
```

```
StopIteration:
```

2

함수의 개념과 구조



2 함수의 개념과 구조

● 함수(Function)란?

- 특정작업을 수행하도록 정의된 코드의 모음
- 입력을 받고 명령을 수행하여 얻은 결과를 반환

함수를 사용하는 이유

- 코드의 재사용성 증가
- 프로그램 구조의 개선
- 유지 보수 및 디버깅 용이

2 함수의 개념과 구조

● 함수의 기본 구조

- 함수 이름 / 매개변수(선택적) / 함수 본문 / 반환값(선택적)
- 매개변수(parameter) : 함수 정의에서 사용하는 변수명
- 인수(argument) : 함수 호출 시 실제로 전달하는 값

```
def 함수이름(매개변수):  
    함수 본문  
    return 반환값
```

2 함수의 개념과 구조

● 함수의 기본 구조

```
[3]: def add_fcn(par1, par2):  
      ans = par1 + par2  
      return ans
```

```
[4]: a = add_fcn(2,3)  
      print(a)
```

5

2 함수의 개념과 구조

● 기본 인수(Default Argument)

```
[7]: def greet(name="손", msg="안녕하세요"):
      print(f"{msg}, {name}님!")
```

```
greet()
greet("영희")
greet("민수", "환영합니다")
```

```
안녕하세요, 손님!
안녕하세요, 영희님!
환영합니다, 민수님!
```

2 함수의 개념과 구조

● 반환값(Return Value)

- `return` : 함수 실행 결과를 반환하고 함수를 종료

```
[1]: def add_fcn(par1, par2):  
      ans = par1 + par2  
      return ans
```

```
[3]: def greet(name="손", msg="안녕하세요"):  
      print(f"{msg}, {name}님!")
```

2 함수의 개념과 구조

● 반환값(Return Value)

- `return` : 함수 실행 결과를 반환하고 함수를 종료

```
[10]: a = add_fcn(2,3)
      b = greet()
```

안녕하세요, 손님!

```
[11]: print(a)
      print(b)
```

5

None

2 함수의 개념과 구조

- 가변 매개변수(Variable-length Parameters)
 - `*args` : 임의의 위치 인수들을(positional arguments) 받음
 - `**kwargs` : 임의의 키워드 인수들을(keyword arguments) 받음

```
[37]: def add_all(*args):  
      return sum(args)
```

```
[38]: print(add_all(1, 2, 3, 4, 5))
```

```
15
```

2 함수의 개념과 구조

● 가변 매개변수(Variable-length Parameters)

- `*args` : 임의의 위치 인수들을(positional arguments) 받음
- `**kwargs` : 임의의 키워드 인수들을(keyword arguments) 받음

```
[39]: def introduce(**kwargs):
      for key, value in kwargs.items():
          print(f"{key}: {value}")
```

```
[40]: introduce(name="Alice", age=25, job="Developer")
```

```
name: Alice
age: 25
job: Developer
```

2 함수의 개념과 구조

● 가변 매개변수(Variable-length Parameters)

- 참고: 키워드 인수는 덕셔너리가 아니다!

```
[6]: my_dic = {"name": "Alice", "age": 25, "job": "Developer"}
      my_dic.items()
```

```
[6]: dict_items([('name', 'Alice'), ('age', 25), ('job', 'Developer')])
```

```
[9]: introduce(my_dic)
```

```
-----
TypeError
Cell In[9], line 1
----> 1 introduce(my
TypeError: introduce
```

```
[12]: introduce(**my_dic)
```

```
name: Alice
age: 25
job: Developer
```


2 함수의 개념과 구조

● 가변 매개변수(Variable-length Parameters)

- `*args`: 임의의 위치 인수들을(positional arguments) 받음
- `**kwargs`: 임의의 키워드 인수들을(keyword arguments) 받음

```
[41]: def example_function(*parameters, **options):  
      print("위치 인수:", parameters)  
      print("키워드 인수:", options)  
  
      example_function(1, 2, 3, a=4, b=5)
```

위치 인수: (1, 2, 3)

키워드 인수: {'a': 4, 'b': 5}

2 함수의 개념과 구조

● 변수의 범위(Scope)

- 지역변수(local variable) : 함수내에서 선언된 변수, 밖에서 접근 불가
- 전역변수(global variable) : 전역(모든 곳)에서 사용 가능한 변수

```
[51]: def add_fcn(par1, par2):  
      ans = par1 + par2  
      return ans  
  
      add_fcn(2,3)
```

```
[51]: 5
```

2 함수의 개념과 구조

● 변수의 범위(Scope)

- 지역변수(local variable) : 함수내에서 선언된 변수, 밖에서 접근 불가
- 전역변수(global variable) : 전역(모든 곳)에서 사용 가능한 변수

```
[52]: print(ans)
```

```
-----  
NameError
```

```
Cell In[52], line 1
```

```
----> 1 print(ans)
```

```
NameError: name 'ans' is not defined
```

2 함수의 개념과 구조

● 변수의 범위(Scope)

- `global` : 함수내에서 전역변수 선언

```
[53]: def add_fcn(par1, par2):  
      global ans  
      ans = par1 + par2  
      return ans  
  
      add_fcn(2,3)
```

```
[53]: 5
```

```
[54]: print(ans)
```

```
5
```

2 함수의 개념과 구조

● 변수의 범위(Scope)

```
[61]: def add_fcn(par1, par2):  
      ans = par1 + par2  
      return ans  
  
      par1 = 3  
      par2 = 4  
      add_fcn()
```

2 함수의 개념과 구조

● 변수의 범위(Scope)

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[61], line 7  
      5 par1 = 3  
      6 par2 = 4  
----> 7 add_fcn()  
  
TypeError: add_fcn() missing 2 required positional arguments: 'par1' and 'par2'
```

2 함수의 개념과 구조

● 변수의 범위(Scope)

```
[60]: def add_fcn(par1, par2):  
      ans = var1 + var2  
      return ans  
  
      var1 = 3  
      var2 = 4  
      add_fcn(2,3)
```

```
[60]: 7
```

3

함수 응용 예제



3 함수 응용 예제

● 람다 함수(Lambda Function)

- 한 줄로 정의되는 익명 함수

```
[43]: multiply = lambda a, b: a * b  
      print(multiply(3, 4))  
  
      12
```

3 함수 응용 예제

● 재귀 함수(Recursive Function)

- 자기자신을 호출하는 함수

```
[64]: def factorial(n):  
        if n == 1:  
            return 1  
        else:  
            return n * factorial(n-1)  
  
print(factorial(5))
```

120

3 함수 응용 예제

● 재귀 함수(Recursive Function)

- 자기자신을 호출하는 함수

```
[83]: def fibonacci(n):  
        if n <= 1:  
            return n  
        else:  
            return fibonacci(n - 1) + fibonacci(n - 2)  
  
        for i in range(10):  
            print(fibonacci(i), end=' ')
```

0 1 1 2 3 5 8 13 21 34

3 함수 응용 예제

● 중첩 함수(Nested Function)

- 함수내에서 정의된 다른 함수
- 클로저(Closure): 외부함수의 변수에 접근 가능한 내부 함수

```
[63]: def outer_function():  
      x = 10 # 외부 함수의 변수  
      def inner_function():  
          print("내부 함수에서 외부 변수 x:", x)  
      inner_function() # 내부 함수 호출  
  
x = 20  
outer_function()
```

내부 함수에서 외부 변수 x: 10

3 함수 응용 예제

● 고차 함수(Higher-order Function)

- 함수를 인수로 받거나 함수를 반환하는 함수

```
[30]: def apply_function(func, value):  
        return func(value)  
  
        def square(x):  
            return x * x  
  
        result = apply_function(square, 4)  
        print(result)
```

16

3 함수 응용 예제

● 고차 함수(Higher-order Function)

- 함수를 인수로 받거나 함수를 반환하는 함수

```
[51]: def make_multiplier(n):  
      def multiplier(x):  
          return x * n  
      return multiplier  
  
double = make_multiplier(2)  
triple = make_multiplier(3)  
print(double(5))  
print(triple(5))
```

10

15

3 함수 응용 예제

● 고차 함수(Higher-order Function)

- `nonlocal` : 현재 함수보다 상위 함수의 변수 사용

```
[81]: def create_counter():  
        count = 0  
        def counter():  
            nonlocal count  
            count += 1  
            print(f"함수가 {count}번 호출되었습니다.")  
            return count  
        return counter
```

3 함수 응용 예제

● 고차 함수(Higher-order Function)

- `nonlocal` : 현재 함수보다 상위 함수의 변수 사용

```
[82]: my_counter = create_counter()
```

```
my_counter()
```

```
my_counter()
```

```
my_counter()
```

함수가 1번 호출되었습니다.

함수가 2번 호출되었습니다.

함수가 3번 호출되었습니다.

```
[82]: 3
```


3 함수 응용 예제

● 고차 함수(Higher-order Function)

- `nonlocal` : 현재 함수보다 상위 함수의 변수 사용

```
[83]: my_counter2 = create_counter()
```

```
my_counter2()
```

```
my_counter()
```

```
my_counter2()
```

함수가 1번 호출되었습니다.

함수가 4번 호출되었습니다.

함수가 2번 호출되었습니다.

```
[83]: 2
```

3 함수 응용 예제

● 데코레이터(Decorator)

- 기존 함수에 기능을 추가하거나 수정할 때 사용되는 함수
- 함수를 감싸는(wrapper) 역할을 함

```
[47]: def my_decorator(func):  
        def wrapper(*args, **kwargs):  
            print("Before function call")  
            func(*args, **kwargs)  
            print("After function call")  
        return wrapper
```

3 함수 응용 예제

● 데코레이터(Decorator)

- 기존 함수에 기능을 추가하거나 수정할 때 사용되는 함수
- 함수를 감싸는(wrapper) 역할을 함

```
[48]: @my_decorator  
def say_hello():  
    print("Hello!")
```

```
[49]: say_hello()  
  
Before function call  
Hello!  
After function call
```

3 함수 응용 예제

● 제너레이터(Generator)

- 반복자(iterator)를 생성해주는 함수로 yield 키워드 사용

```
[49]: def countdown(n):  
        while n > 0:  
            yield n  
            n -= 1  
  
count3 = countdown(3)
```

3 함수 응용 예제

● 제너레이터(Generator)

- 반복자(iterator)를 생성해주는 함수로 yield 키워드 사용

```
[52]: next(count3)
```

```
[52]: 3
```

```
[53]: next(count3)
```

```
[53]: 2
```

```
[54]: next(count3)
```

```
[54]: 1
```

3 함수 응용 예제

● 제너레이터(Generator)

- 반복자(iterator)를 생성해주는 함수로 yield 키워드 사용

```
[55]: next(count3)
```

```
-----  
StopIteration
```

```
Cell In[55], line 1
```

```
----> 1 next(count3)
```

```
StopIteration:
```

4

클래스



4

클래스

● 클래스(Class)

- 객체(object)를 생성하기 위한 설계도
- 속성과 메서드로 특정 개념을 표현

클래스의 주요 개념

- 속성(attribute): 클래스에 정의된 변수(데이터)
- 메서드(method): 클래스에 정의된 함수
- 객체(object): 클래스로 만들어진 실체(인스턴스)

4

클래스

● 클래스(Class)

```
[84]: class Car:
        def __init__(self, color, power, speed=0):
            self.color = color
            self.power = power
            self.speed = speed
        def forward(self):
            self.speed = self.speed + self.power*10
            print(f"{self.color} 자동차가 {self.speed}km/h로 달립니다.")
        def backward(self):
            self.speed = self.speed - self.power*10
            print(f"{self.color} 자동차가 {self.speed}km/h로 달립니다.")
```

4

클래스

● 클래스(Class)

```
[101]: car1 = Car("빨간색",1)
       car2 = Car("파란색",2)
       car1.forward()
       car1.forward()
       car1.backward()
       car1.backward()
       car2.forward()
       car2.forward()
```

빨간색 자동차가 10km/h로 달립니다.
빨간색 자동차가 20km/h로 달립니다.
빨간색 자동차가 10km/h로 달립니다.
빨간색 자동차가 0km/h로 달립니다.
파란색 자동차가 20km/h로 달립니다.
파란색 자동차가 40km/h로 달립니다.

4 클래스

● 클래스(Class)

```
[98]: print(car1.color)
      print(car1.speed)
      print(car1.power)
```

빨간색

0

1

```
[99]: car1.power = 4
```

```
[100]: car1.forward()
       car1.forward()
```

빨간색 자동차가 40km/h로 달립니다.

빨간색 자동차가 80km/h로 달립니다.

✧ 정리하기

■ 반복자

반복 가능 객체에서 요소를 하나씩 꺼내주는 객체

■ 함수

특정 작업을 수행하는 코드 블록, def 키워드를 사용하여 정의

■ 변수의 범위

지역변수는 함수 내에서, 전역변수는 프로그램 전체에서

■ 람다함수 / 재귀함수

한 줄로 표현되는 익명함수 / 자기 자신을 호출하는 함수

■ 클래스

객체를 생성하기 위한 설계도

03 강

다음시간안내

파이썬과 머신러닝

